Reinforcement learning from human feedback

SESSION 1 · FROM BASE MODELS TO RLHF

Pegah Alizadeh

pegahani.github.io

Master's course, 2026–2027

Welcome

This course is about **how language models are trained to follow instructions and to match human preferences**, and about the statistics behind it.

By the end of the course, you will be able to:

- **Explain the pipeline:** instruction tuning, reward models, DPO and RL, with the theory behind each step
- **Model preferences:** fit and evaluate models of human comparisons, with honest uncertainty
- **Train models yourself:** small language models, on free GPUs
- **Do research:** design, run and report a small project of your own

The course at a glance

Session	When	Topic	In class
1	Week 1	From base models to RLHF	Lab: instruction tuning
2	Week 2	Preferences and reward models	Lab: label outputs, train a reward model
3	Week 2	DPO and its noise-aware variants	Lab: a small DPO run
4	Week 2	RL for language models	Lab: policy gradients for language models
5	Week 4	Over-optimization, AI feedback, open problems	Project feedback

Each session lasts 3 hours: a lecture with theory, a short break, then a lab of about an hour.

Where the materials come from

Two books, read together:

- **Lambert, *Reinforcement Learning from Human Feedback*** (rlhfbook.com): training language models in practice
- **Truong, Haupt, Koyejo, *Machine Learning from Human Preferences*** (mlhp.stanford.edu): the statistics of preferences

Also:

- **Slides:** adapted from Lambert's lectures (rlhfbook.com/course), with material from the second book and new derivations
- **Labs:** adapted from the code that accompanies Lambert's book
- **Homework:** written for this course, with its sources cited in each notebook

Figures from papers keep their citations. Any errors are mine.

Reading plan

Session	Lambert, *RLHF*	Truong et al., *Machine Learning from Human Preferences*
1	Ch. 1, 3, 4	Introduction; Sections 1.1 and 1.2
2	Ch. 5, 10, 11	Ch. 1; Section 2.3
3	Ch. 8, 9	Ch. 2: preference learning for language models
4	Ch. 6, 7	Ch. 3: bandits
5	Ch. 12, 14, 15	Ch. 4 and 5: behavioral biases, aggregation

Further reading: T. Kaufmann et al., "A survey of reinforcement learning from human feedback", TMLR 2025.

How we work

- **Lecture, then lab:** each lab runs in your browser, on a free GPU. Work in pairs.
- **Homework at three levels:**
  - **Foundation:** everyone can complete it, with hints and solutions
  - **Core:** the level we aim for
  - **Deep:** derivations and open questions, if you want to go further
- **Low entry, earned depth:** you don’t need any experience with language models. It’s easy to get started, and the depth comes from doing the exercises.

The project

Your grade is based on the project only. Homework isn't graded, but it prepares you for the project.

Teams of 2 or 3, on a topic you choose, from a wide range of possibilities. We discuss ideas in session 4.

When	What
End of session 4	Discuss your idea with me
A few days later	One-page proposal: question, hypothesis, method, baseline, compute plan
Session 5	Early results and feedback
About 4 weeks after session 5	Report, code, and a 10-minute video
The week after	A 15-minute oral exam for each team

What counts: a clear question, a baseline, several runs with intervals, and honest reporting. A well-run negative result can earn full marks.

Class materials

What	Where
Main books	rlhfbook.com and mlhp.stanford.edu
Lambert's lectures and videos	rlhfbook.com/course
Book code	github.com/natolambert/rlhf-book
Labs and homework	github.com/pegahani/rlhf-course
Free GPUs	Google Colab or Kaggle: choose a T4 GPU
Gentle first reading	Illustrating RLHF, Hugging Face blog
If language models are new to you	Hugging Face LLM course

For today's lab: a Google or Kaggle account and a laptop.

Today

1. **Language models in practice:** the few tools we need
2. **The big picture:** what RLHF is and where it came from
3. **Theory:** from RL to the RLHF objective, and its optimal solution
4. **Instruction tuning:** chat templates and the prompt mask
5. **Lab:** turn a base model into an assistant

You know RL and statistics already, so we'll spend our time on what's new: **language models, and what changes when the reward comes from people.**

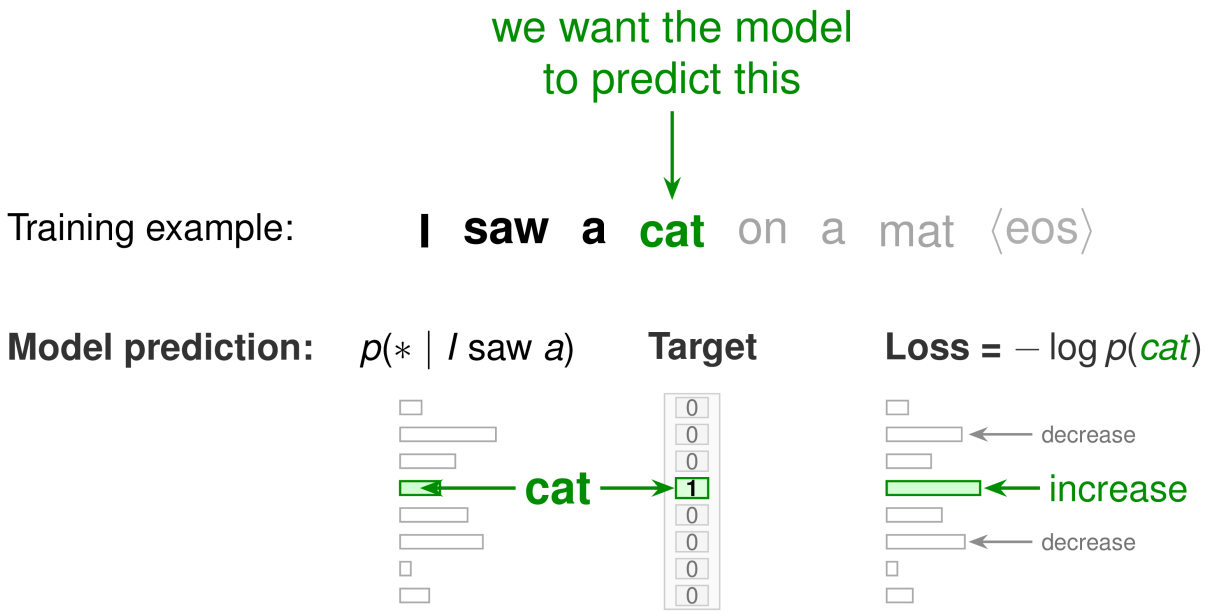
Language models in practice

What a language model is

A language model assigns **probabilities to text**, one **token** (a word or part of a word) at a time. By the chain rule,

$$P_\theta(x) = \prod_{t=1}^{T} P_\theta(x_t \mid x_1, \ldots, x_{t-1}).$$

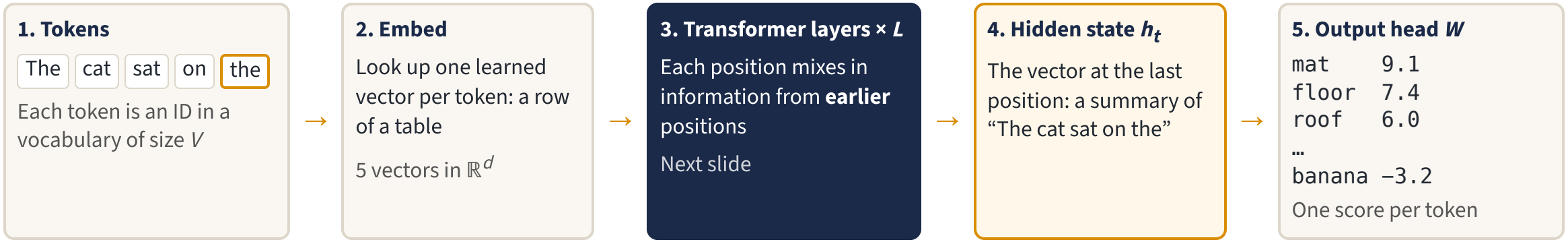
This is called **autoregressive**. For this course, the model is a function from the tokens so far to next-token probabilities. Next, we open it once.



Next-token prediction: a decoder-only language model predicts one token at a time from the prefix before it.

Vaswani et al., 2017

Inside the model: one prediction, step by step



Steps 1 to 4 learn features; step 5 is a linear classifier on them. The head scores every token v with a dot product, $z_v^{(t)} = w_v^\top h_t$, so $z^{(t)} = W h_t \in \mathbb{R}^V$. The scores above are illustrative.

> ***For statisticians:*** *multinomial logistic regression with V classes, on features h_t that the network learns.*

The lab's model, OLMo-2 1B: $V \approx 100\text{K}, d = 2{,}048, L = 16$. We train only the last 4 layers and the output head.

Vaswani et al., 2017

What one transformer layer does

Each layer updates every position's vector in two steps.

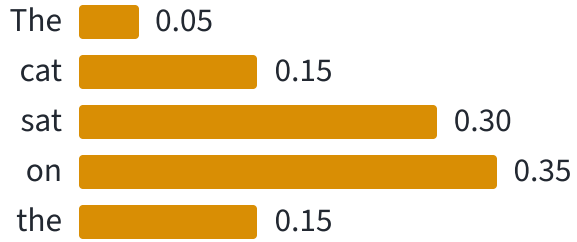
1. Attention: look back. Position t adds a weighted average of earlier positions:

$$h_t \leftarrow h_t + \sum_{s \le t} \alpha_{ts}\, v_s, \qquad \alpha_{ts} = \frac{e^{q_t^\top k_s}}{\sum_{s' \le t} e^{q_t^\top k_{s'}}}.$$

q_t, k_s, v_s are linear maps of the vectors: what t **looks for**, what s **offers**, what s **passes on**.

2. MLP: process locally. A small network transforms each position on its own.

Attention weights of the last "the" (illustrative)



"sat on the" is what makes "mat" likely, so a useful layer looks there. The weights are a softmax, and $s \le t$ is the causal mask.

> ***For statisticians:*** *attention is kernel smoothing, as in Nadaraya-Watson, with a similarity kernel learned from data.*

Vaswani et al., 2017

From scores to log-probabilities

The model outputs one score per vocabulary token, called a **logit** $z^{(t)}$. **Softmax** turns these into a distribution:

$$P_\theta(x_t \mid x_{<t}) = \frac{e^{z^{(t)}_{x_t}}}{\sum_j e^{z^{(t)}_j}}.$$

A sequence is a product of these terms, so in log-space it's a sum:

$$\log P_\theta(x) = \sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t}).$$

Log-probabilities are numerically stable, and sums are easy to differentiate. **You'll compute exactly this in today's lab**, and DPO in session 3 is built from it.

One training example, as tensors

```
logits = model(input_ids).logits          # (B, L, V): a score per vocabulary token and position
logits = logits[:, :-1, :]                # drop the last position: it has no next token to predict
labels = input_ids[:, 1:]                 # the target at each position is the actual next token
completion_mask = completion_mask[:, 1:]  # shift the mask in the same way (1 = response token)
log_probs = logits.log_softmax(dim=-1)    # normalize into log-probabilities over the vocabulary
token_lp = log_probs.gather(-1, labels.unsqueeze(-1)).squeeze(-1)  # log-prob of each true next token
seq_lp = (token_lp * completion_mask).sum(dim=-1)                  # one log-prob per sequence: (B,)
```

B is the batch size, L the sequence length and V the vocabulary size.

The **one-token shift** is the autoregressive step made literal: position t's logits predict token $t + 1$.

Three masks

Masks are where silent bugs live in language model code. Three different masks do three different jobs:

- **Causal mask:** position t can't attend to later positions. It's built into the model, so you won't touch it.
- **Attention (padding) mask:** tells the model to ignore padding when sequences of different lengths share a batch.
- **Completion (loss) mask:** only response tokens count in the loss. **The prompt conditions the prediction but receives no loss.**

If the completion mask is wrong, you train on the prompt or on padding, and nothing crashes.

Training is maximum likelihood

With a one-hot label on the true token x_t, cross-entropy at one position collapses to one term:

$$H(q_t, P_\theta) = -\sum_v q_t(v) \log P_\theta(v \mid x_{<t}) = -\log P_\theta(x_t \mid x_{<t}).$$

Summing over positions and averaging over data gives the negative log-likelihood:

$$\mathcal{L}_{\mathrm{LM}}(\theta) = -\mathbb{E}_{x \sim \mathcal{D}} \left[\sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t}) \right].$$

Cross-entropy is a **proper scoring rule**: in expectation, it's minimized by the true probabilities. Pretraining aims at calibrated prediction, but prediction alone doesn't make a model helpful (Truong et al., 2025).

Pretraining, then post-training

- **Pretraining:** next-token cross-entropy on massive amounts of raw text. **Every** token contributes to the loss.
- **Instruction tuning (SFT):** the same cross-entropy, on instruction and response pairs. **Only the response** contributes to the loss.
- **Preference tuning (RLHF):** response-level, contrastive feedback.
- **RL with verifiable rewards (RLVR):** rewards from a checker, such as a correct math answer.

Together, these steps after pretraining are called **post-training**. They share one goal:

> *Change the model's distribution over responses toward behavior we want.*

The big picture

A base model completes text

After pretraining we have a very capable autocomplete:

USER

The president of the United States in 2006 was

ASSISTANT (Llama 3.1 405B Base)

George W. Bush, the governor of Florida in 2006 was Jeb Bush, and John McCain was an Arizona senator in 2006…

Post-training makes it answer

The same prompt, after post-training:

USER

The president of the United States in 2006 was

ASSISTANT (Tülu 3 405B)

George W. Bush was the president of the United States in 2006. He served two terms in office, from January 20, 2001, to January 20, 2009.

What each training stage changes

- **Pretraining:** builds world knowledge, language fluency and broad capabilities
- **Instruction tuning:** teaches the question-and-answer format
- **Preference tuning:** uses a contrastive loss to change completions as a whole
- **RLVR:** improves the model on questions whose answers can be checked

A question to keep in mind, and to test in the lab: is post-training "just style transfer", or does it change what the model can do? (Zhou et al., 2023)

Why RLHF?

In classical RL the reward is known: CartPole gives +1 per step. For many tasks it's **hard to write down**:

- It's easy to judge which poem is better, but hard to write a formula for "better"
- It's easy to spot a helpful answer, but hard to define "helpful"
- The most likely continuation of a text isn't always the most useful one

RLHF lets us optimize for behavior we can **evaluate**, even when we can't **specify** the reward.

Christiano et al., 2017; Ouyang et al., 2022; Ziegler et al., 2019

Preferences are everywhere

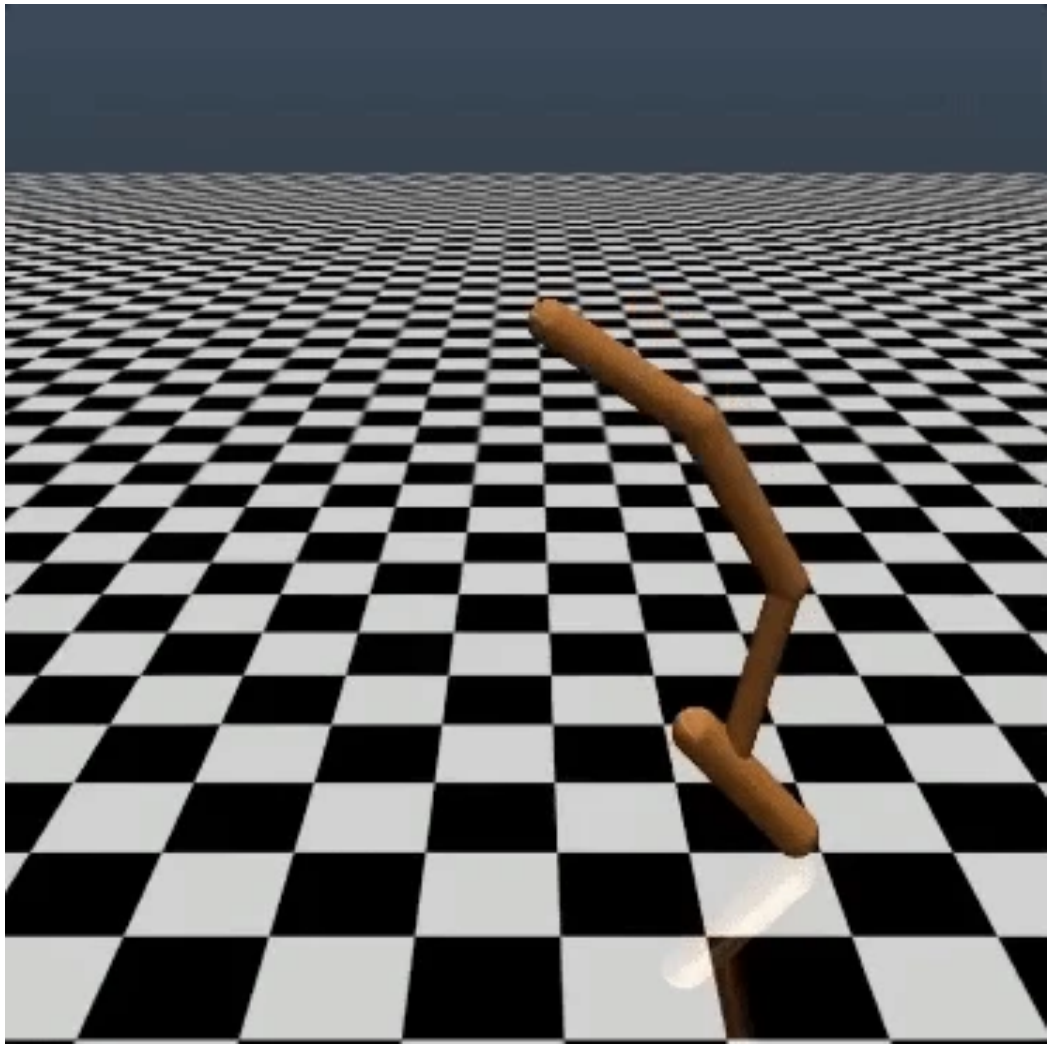
Language models are one example of a much older problem: **learning from comparisons**.

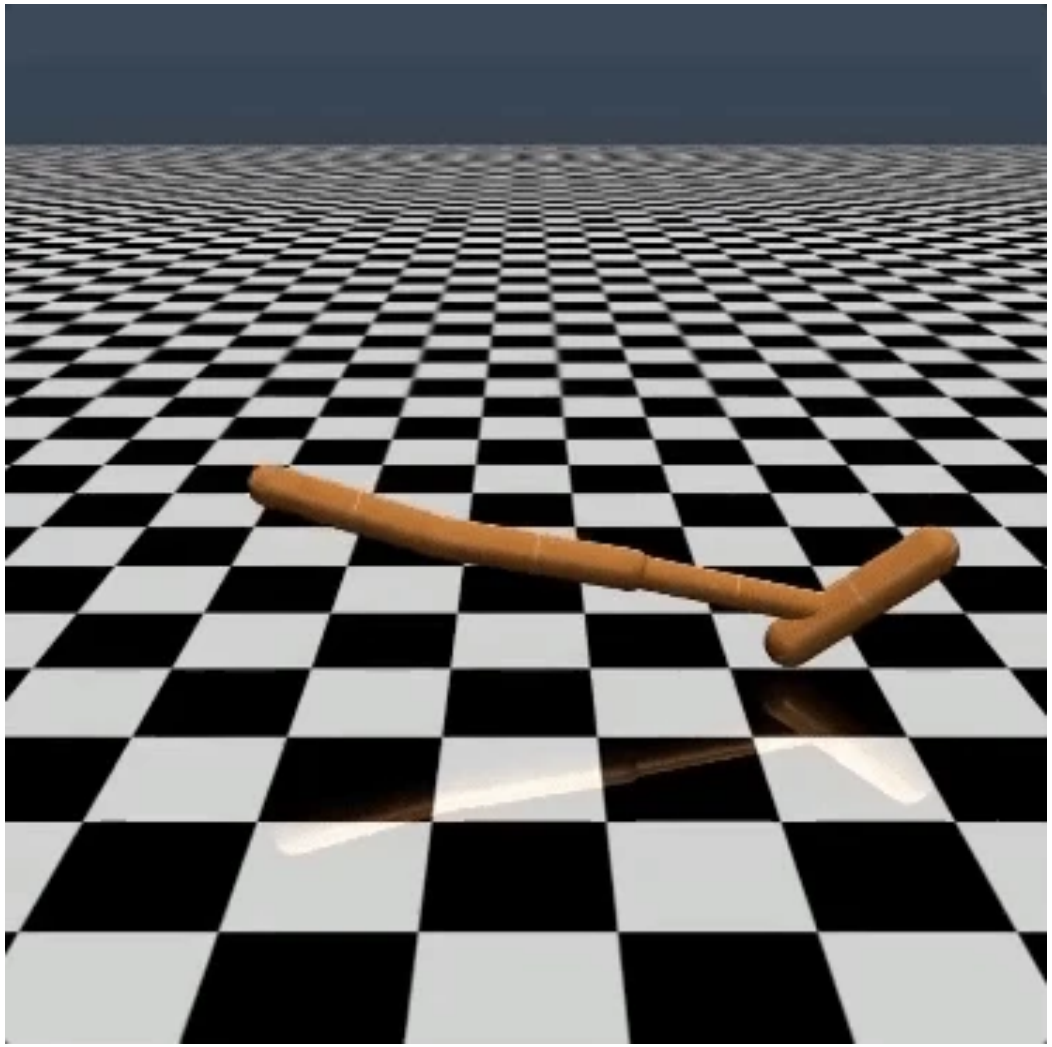
- **Recommender systems:** a click on item A rather than B suggests A > B
- **Search:** clicking the third result suggests it beat the first two
- **Robotics:** people prefer smooth trajectories to jerky ones
- **Games and sports:** each match is a comparison, and Elo ratings summarize them
- **Language models:** annotators judge which response is more helpful, harmless or honest

These share **one mathematical structure**: comparisons or choices that reveal underlying preferences. The Bradley-Terry model and its extensions give a common framework for all of them.

Truong et al., 2025

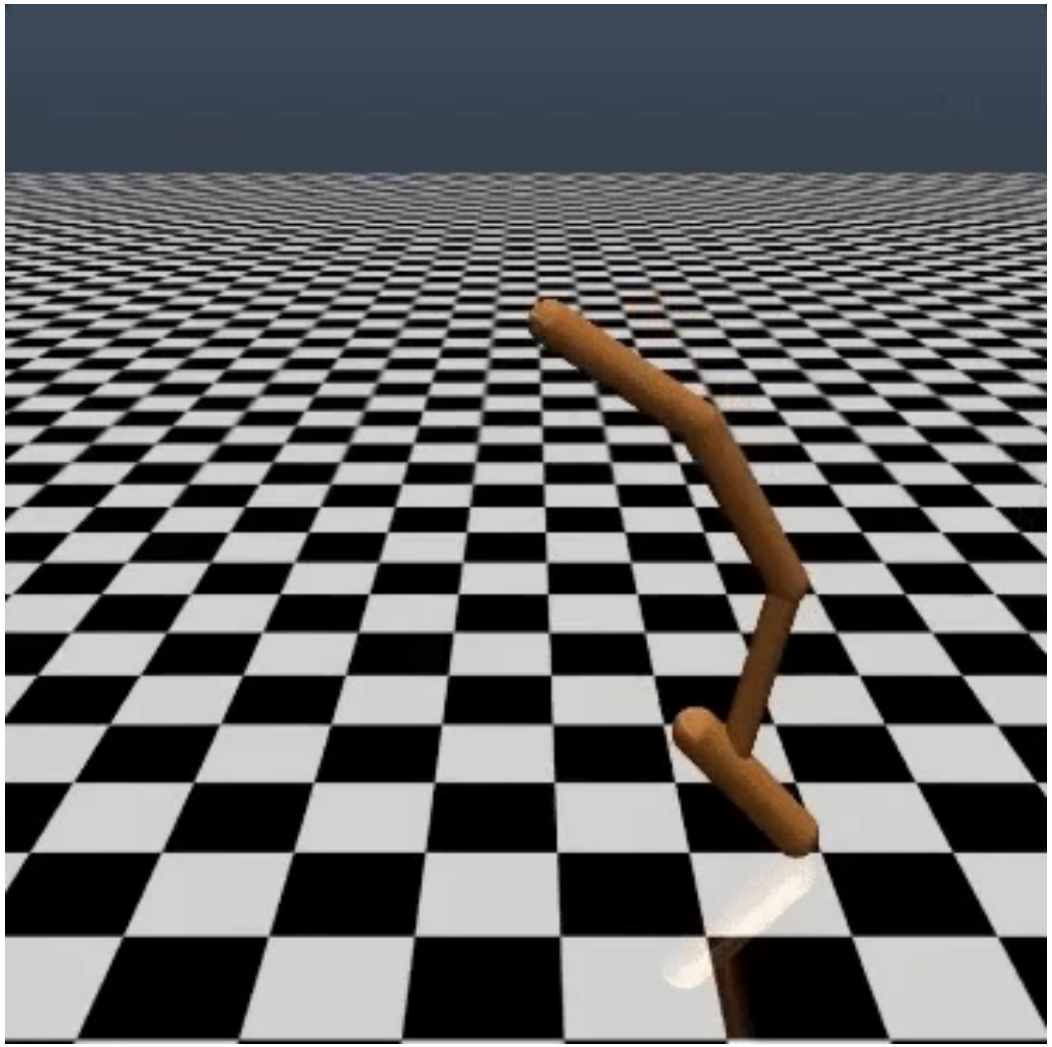
Which is the better backflip?

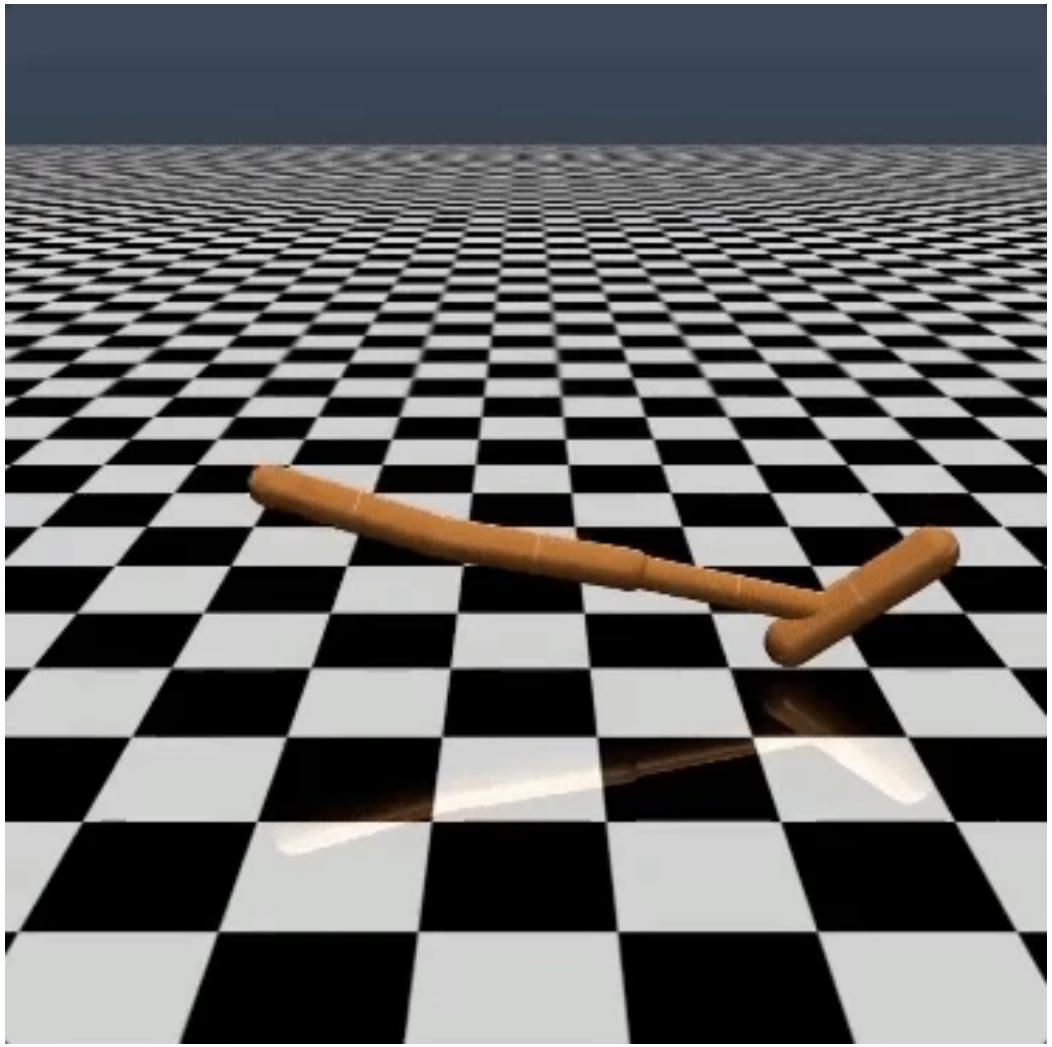




Christiano et al., 2017

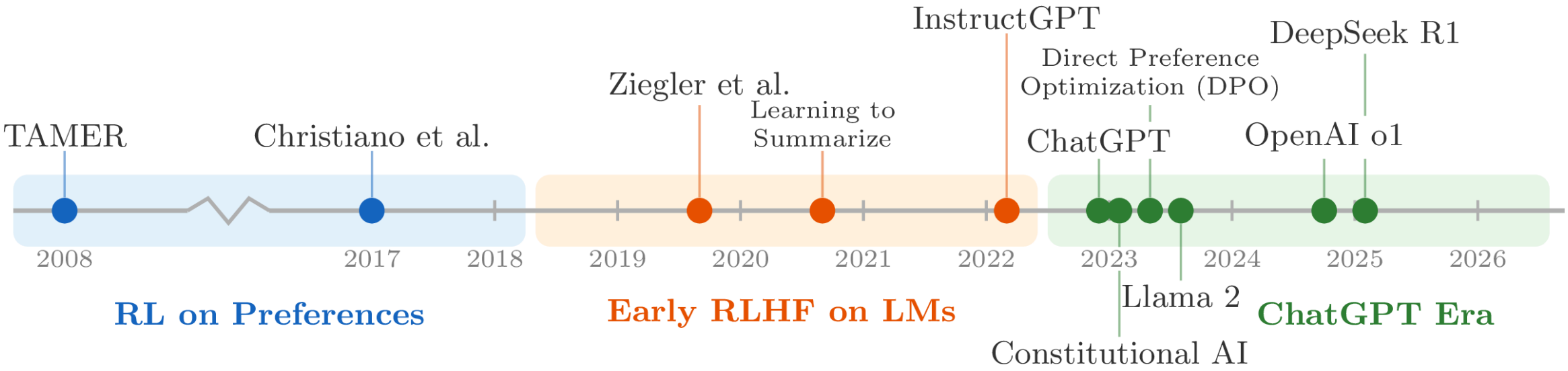
Left: human feedback; right: hand-designed reward





Christiano et al., 2017

The path to modern RLHF



- **Ziegler** (Ziegler et al., 2019): RLHF on language models
- **InstructGPT** (Ouyang et al., 2022): canonical recipe
- **Constitutional AI** (Bai et al., 2022): AI feedback
- **DPO** (Rafailov et al., 2023): no separate reward model
- **Tülu 3** (Lambert et al., 2024): multi-stage recipe
- **DeepSeek R1** (Guo et al., 2025): popularized RLVR

What comparison data looks like

An annotator, human or AI, sees two responses to the same prompt and picks the better one.

USER

Explain why the sky is blue in one sentence.

ASSISTANT (Response A ✓)

The sky is blue due to Rayleigh scattering, where shorter blue wavelengths of sunlight are scattered more by atmospheric molecules than longer wavelengths.

USER

Explain why the sky is blue in one sentence.

ASSISTANT (Response B)

The sky appears blue because of the way light interacts with the atmosphere and stuff, it's basically just physics.

Types of comparison data

Preference data comes in several forms, for items $j \in \{1, \ldots, M\}$:

- **Full rankings:** $j_1 \succ j_2 \succ \cdots \succ j_M$. These are informative, but hard for people to give.
- **Choice from a set:** $(j, \mathcal{S})$, meaning j is preferred to everything else in $\mathcal{S}$
- **Binary comparisons:** $\mathcal{S} = \{j, j'\}$, written $Y_{jj'} = 1$. These are cheap and quick to collect, and RLHF uses them most.
- **Accept or reject:** a comparison with an outside option, "nothing"

Data can also depend on a **context**, such as a user, a goal or a prompt. In RLHF, one example is a triple (x, y_w, y_l): a prompt, a preferred response and a rejected one.

Truong et al., 2025

Why model preferences as random?

Data is noisy: the same person can answer differently on another day, and different people disagree. So we treat a preference as a **random order** over the items, with some distribution $p(\prec)$.

The usual assumptions are strong but practical:

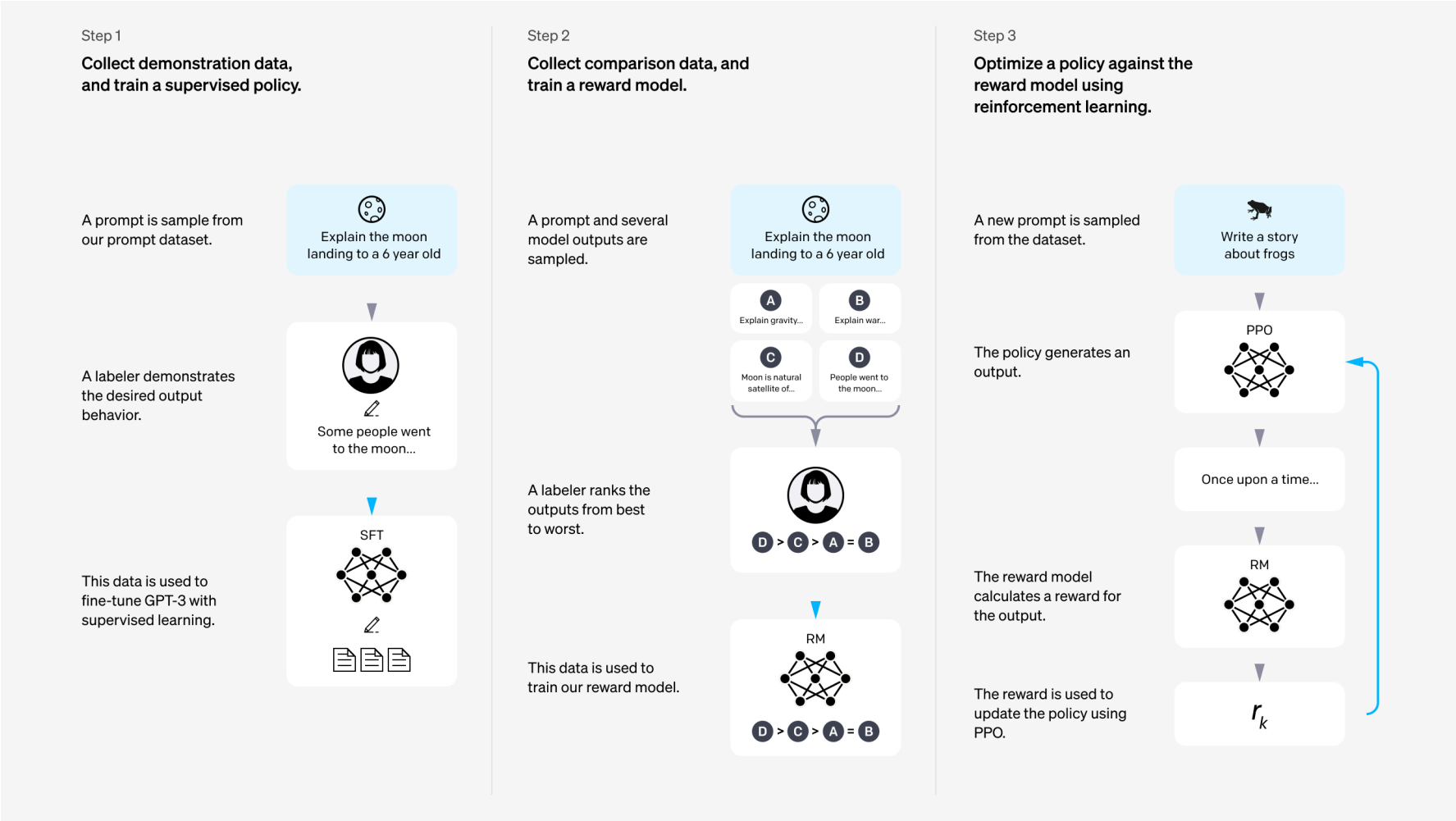
- **Totality:** any two items can be compared
- **Transitivity:** if $j \succ j'$ and $j' \succ k$, then $j \succ k$

A general distribution over orders of M items has $M! - 1$ parameters: 23 for $M = 4$, and over 3.6 million for $M = 10$. We need structure, and Bradley-Terry is the simplest.

People's preferences can also be **incomplete**, for example when helpfulness and safety pull in different directions. That's an absence of preference, not randomness.

Truong et al., 2025

InstructGPT's three-step recipe



The three-step RLHF process from InstructGPT.

Ouyang et al., 2022

Step 1: instruction fine-tuning

Start from a pretrained model and collect demonstrations of the desired behavior. Then use standard supervised learning on the response tokens:

$$\mathcal{L}_{\mathrm{SFT}}(\theta) = -\sum_{(x,y^\star)} \sum_{t=1}^{|y^\star|} \log \pi_\theta \left(y_t^\star \mid x, y_{<t}^\star\right)$$

The notation changes here: the model is now a **policy** π_θ, x is the prompt and $y^\star$ the demonstrated response.

This is today's lab.

Ouyang et al., 2022

Step 2: reward modeling

Collect comparisons between two responses to the same prompt, then train a reward model $r_\phi(x, y)$ to score the preferred one higher:

$$P(y_w \succ y_l \mid x) = \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)$$

$$\mathcal{L}_{\mathrm{RM}}(\phi) = -\log \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)$$

Here y_w is the winning response and y_l the losing one.

As statisticians, you may recognize this model: it's **Bradley-Terry**, and it's the subject of session 2.

Christiano et al., 2017; Ouyang et al., 2022

A worked example

Three responses with rewards $V_A = 2, V_B = 1$ and $V_C = 0$. Under Bradley-Terry, $P(j \succ k) = \sigma(V_j - V_k)$:

Comparison	Reward difference	Probability
$A \succ B$	1	$\sigma(1) \approx 0.731$
$A \succ C$	2	$\sigma(2) \approx 0.881$
$B \succ C$	1	$\sigma(1) \approx 0.731$

For a full ranking, the **Plackett-Luce** model picks the winner, then the winner among the rest:

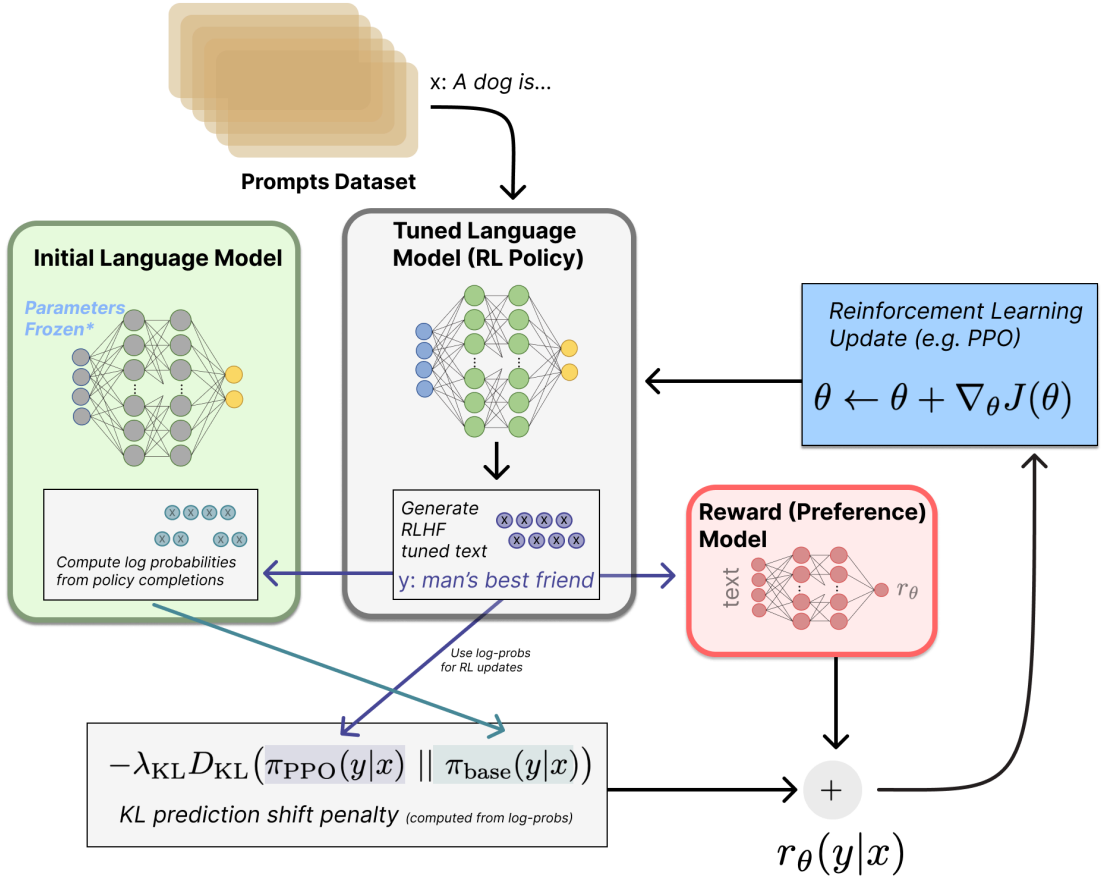
$$P(A \succ B \succ C) = \frac{e^2}{e^2 + e^1 + e^0} \cdot \frac{e^1}{e^1 + e^0} \approx 0.665 \times 0.731 \approx 0.486.$$

Only **differences** of rewards matter: adding a constant to every V changes nothing.

Truong et al., 2025

Step 3: RL against the reward model

- Sample prompts x_i from the dataset
- Generate responses $y_i \sim \pi_\theta(\cdot \mid x_i)$
- Score them with the reward model $r_\phi(x_i, y_i)$
- Add a **KL penalty** to stay close to the reference model
- Update the policy with a policy-gradient algorithm (PPO in InstructGPT)



Ouyang et al., 2022

Three ways to optimize preferences

	Rejection sampling	Online RL (PPO)	DPO
Mechanism	Filter, then SFT	Generate, score, update	Direct gradient on preferences
Reward model	Required	Required	Implicit
On-policy data	Yes	Yes	No (fixed dataset)
Complexity	Low	High	Low

They share an objective and differ in **how** they move the policy toward higher reward. We'll study each of them in sessions 3 and 4.

Caveat: the reward is a proxy

A reward model is only *correlated* with what people want. **Goodhart's law:** "When a measure becomes a target, it ceases to be a good measure."

What this looks like in practice:

- **Reward hacking:** the score keeps climbing while actual quality falls
- **Verbosity bias:** longer responses score higher, so models get verbose
- **Sycophancy:** the model tells users what they want to hear
- **Over-refusal:** the model refuses reasonable requests

We return to this in session 5.

Gao et al., 2023

Theory: from RL to the RLHF objective

What changes from standard RL

	Standard RL	RLHF for language models
Policy	Often learned from scratch	Fine-tuned from a pretrained model
Reward	Environment function $r(s_t, a_t)$	Learned model $r_\phi(x, y)$
Transitions	Dynamics $p(s_{t+1} \mid s_t, a_t)$	Usually none: prompts come from a dataset
Action	One environment action	A completion y, a sequence of tokens
Reward timing	Often per step	Usually once, for the whole response
Horizon, discount	$T > 1, \gamma < 1$	Often single-step, $\gamma = 1$

Source: Chapter 3, “Manipulating the standard RL setup”.

Two views of the same problem

Token-level view (an MDP). The state is the prompt plus the tokens so far, the action is the next token, and the transition appends it deterministically.

$$\pi_\theta(y \mid x) = \prod_t \pi_\theta(y_t \mid x, y_{<t})$$

Response-level view (a contextual bandit). The context is the prompt x, the action is the whole response y, and there is one reward.

$$\max_\pi \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi(\cdot|x)} \left[r_\phi(x, y) \right]$$

The second view is the simpler one for analysis, and we use it next. The action space is enormous, but the problem is well defined.

KL divergence

The KL penalty is central to the course:

$$D_{\mathrm{KL}}(p \,\|\, q) = \sum_i p_i \log \frac{p_i}{q_i}.$$

- It's non-negative, and 0 only when $p = q$
- It's **asymmetric**, and it doesn't satisfy the triangle inequality

Practitioners often say "KL distance". It's a useful shorthand for how much the model has changed, but **KL is a divergence, not a metric**.

Which direction of KL?

The RLHF penalty is $D_{\mathrm{KL}}(\pi_\theta \,\|\, \pi_{\mathrm{ref}}) = \mathbb{E}_{y\sim\pi_\theta}\left[\log \frac{\pi_\theta(y|x)}{\pi_{\mathrm{ref}}(y|x)}\right]$, a **reverse KL**.

- **Easy to estimate:** the expectation is under the policy being trained, so the responses we generate are enough.
- **Mode-seeking:** it's large where π_θ puts probability and π_{ref} puts very little. The policy stays where the reference is plausible, without having to cover all its modes.

You'll meet the other directions too:

- **SFT minimizes a forward KL:** maximum likelihood on data minimizes $D_{\mathrm{KL}}(p_{\mathrm{data}} \,\|\, \pi_\theta)$, which is mass-covering.
- **PPO implementations often monitor $D_{\mathrm{KL}}(\pi_{\mathrm{old}} \,\|\, \pi_\theta)$,** estimated on the batch sampled from π_{old}.

In each case, the expectation is under the distribution we can sample from. Source: Chapter 15.

The RLHF objective

$$\max_{\pi} \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi(\cdot|x)} \quad \underbrace{r_\phi(x,y)}_{\text{maximize the reward}} \quad - \underbrace{\beta D_{\mathrm{KL}}\left(\pi(\cdot \mid x) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid x)\right)}_{\text{but don't change the model too much}}$$

- π_{ref} is the reference model, usually the SFT model
- β sets the trade-off between improving behavior and staying close to what the model already knows
- This matters because we start from a strong pretrained model, not from a random policy

Question: if we could solve this exactly, what would the optimal policy look like?

The optimal policy: set-up

The objective separates across prompts, so fix x and drop it from the notation. We treat responses as a finite set and write $\pi(y)$ for the probabilities:

$$F(\pi) = \sum_y \pi(y)\, r(y) - \beta \sum_y \pi(y) \log \frac{\pi(y)}{\pi_{\text{ref}}(y)} \quad \text{subject to} \quad \sum_y \pi(y) = 1.$$

F is concave in π, because the first term is linear and negative entropy is convex. So a stationary point of the Lagrangian is a global maximum:

$$\mathcal{L}(\pi, \lambda) = F(\pi) + \lambda \left(\sum_y \pi(y) - 1 \right).$$

The optimal policy: derivation

Differentiate with respect to one $\pi(y)$ and set the result to zero:

$$r(y) - \beta \left(\log \frac{\pi(y)}{\pi_{\text{ref}}(y)} + 1 \right) + \lambda = 0.$$

Isolate the log-ratio:

$$\log \frac{\pi(y)}{\pi_{\text{ref}}(y)} = \frac{r(y)}{\beta} + \frac{\lambda - \beta}{\beta}.$$

Exponentiate, and write the constant as $e^{(\lambda-\beta)/\beta} = 1/Z$. The constraint $\sum_y \pi(y) = 1$ then fixes Z:

$$\pi(y) = \frac{1}{Z}\, \pi_{\text{ref}}(y)\, e^{r(y)/\beta}, \qquad Z = \sum_y \pi_{\text{ref}}(y)\, e^{r(y)/\beta}.$$

The optimal policy: what it means

$$\pi^*(y \mid x) = \frac{\pi_{\text{ref}}(y \mid x)\, e^{r(x,y)/\beta}}{Z(x)}$$

The optimal policy **reweights the reference model** exponentially in the reward.

- $\beta \to \infty$: $\pi^* \to \pi_{\text{ref}}$, so nothing changes
- $\beta \to 0$: π^* concentrates on the highest-reward responses in the support of π_{ref}

Why not just use this formula? $Z(x)$ sums over all possible responses, and the reward is itself only an estimate. So we need algorithms that approximate π^*.

A preview of session 3

Take the log of the optimal policy and solve for the reward:

$$r(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x).$$

Now compare two responses to the same prompt. $Z(x)$ doesn't depend on y, so it cancels:

$$r(x, y_w) - r(x, y_l) = \beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}.$$

The intractable constant is gone, and only log-probabilities of the model remain. This is the key idea of DPO (Rafailov et al., 2023): the log-probabilities you compute in today's lab are enough. The implicit reward $\beta \log \frac{\pi_\theta}{\pi_{\text{ref}}}$ plugs straight into Bradley-Terry (Truong et al., 2025).

Instruction tuning

Chat templates

A base model only sees a sequence of tokens. A **chat template** turns a list of messages into one flat sequence:

```
<|im_start|>system
You are a friendly chatbot who always responds in the style of a pirate<|im_end|>
<|im_start|>user
How many helicopters can a human eat in one sitting?<|im_end|>
<|im_start|>assistant
```

- **Roles:** `system` holds instructions, `user` holds the person's messages, and `assistant` holds the model's responses
- **Generation cue:** the final `<|im_start|>assistant` tells the model that it's its turn
- **Stopping:** the model generates until it produces the end token, here `<|im_end|>`

Each model family has its own format. Source: Chapter 4.

What instruction tuning does differently

The loss is the same as pretraining. The differences are in the data and the set-up:

- **Prompt masking:** the model isn't trained to predict user messages, only responses
- **Smaller batches:** to fit a narrower data distribution while keeping what pretraining taught
- **Lower learning rate:** typically one to two orders of magnitude below pretraining

The main effect is simple: **a base model continues text; an instruction-tuned model answers and stops.**

Learning to stop is a good example of something SFT must teach. The target includes the end token, so the model is trained to produce it.

What makes instruction tuning work

Principles from Chapter 4:

- **Data quality is key.** The completions are what the model actually learns from.
- **Small can be enough for chat.** Focused datasets can do well in narrow domains (Zhou et al., 2023). The dataset in today's lab, No Robots, has about 10K human-written samples.
- **Match the use case.** The best prompts resemble the tasks you care about downstream.
- **Later stages can recover from some noise.** The overall recipe matters more than any single stage.

Lab: from a base model to an assistant

Lab: what you'll do

Open the notebook in Colab or Kaggle, and choose a T4 GPU: Open the lab in Colab. All notebooks are at github.com/pegahani/rlhf-course.

1. **Run the base model** on a few prompts
2. **Read the chat template** and write the prompt mask (exercise A)
3. **Compute a response's log-probability** by hand (exercise B)
4. **Train** on 3,000 examples for about 10 minutes, while you discuss the results so far
5. **Compare before and after,** with an interval for the stop rate (exercise C)

Work in pairs, especially if Colab has no GPU available for you. Each exercise has a hint and a solution, so use them if you get stuck.

What we saw

One test run on a free T4, on 32 held-out prompts:

	Before	After
Stopped by itself	5/32	15/32
95% interval	[0.07, 0.32]	[0.31, 0.64]

Paired: 13 prompts stopped only after training, and 3 only before.

How do your numbers compare?

USER
What is the capital of France?

ASSISTANT (After training)
The capital of France is Paris.

USER
Write a haiku about programming.

ASSISTANT (After training)
The best way to learn programming is to do it.

To discuss: did it learn to stop, or also learn something new? Is stopping the same as quality?

Before next session

Homework, due at session 2: both notebooks are at github.com/pegahani/rlhf-course

- **HW0, do it first:** from NumPy to PyTorch, practising the tools from today's lab

HW1: the Bradley-Terry model

- **Foundation:** simulate Bradley-Terry data and fit the model
- **Core:** build confidence intervals and test their coverage
- **Deep:** Fisher information, and coverage when comparisons are correlated

Reading for session 2

- **Lambert, *RLHF*:** Chapter 5 (Bradley-Terry reward models), and Chapters 10 and 11
- ***Machine Learning from Human Preferences*:** Chapter 1, especially Sections 1.2, 1.6 and 1.9, and Section 2.3

References (1/2)

Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., et al. "*Constitutional ai: Harmlessness from ai feedback*." *arXiv preprint arXiv:2212.08073*, 2022.

Christiano, P., Leike, J., Brown, T., Martic, M., Legg, S., et al. "*Deep Reinforcement Learning from Human Preferences*." *Advances in Neural Information Processing Systems*, 2017.

Gao, L., Schulman, J., and Hilton, J. "*Scaling laws for reward model overoptimization*." *International Conference on Machine Learning*, 2023.

Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., et al. "*Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning*." *arXiv preprint arXiv:2501.12948*, 2025.

Lambert, N., Morrison, J., Pyatkin, V., Huang, S., Ivison, H., et al. "*Tülu 3: Pushing Frontiers in Open Language Model Post-Training*." *arXiv preprint arXiv:2411.15124*, 2024.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., et al. "*Training language models to follow instructions with human feedback*." *Advances in Neural Information Processing Systems*, 2022.

Rafailov, R., Sharma, A., Mitchell, E., Manning, C., Ermon, S., et al. "*Direct preference optimization: Your language model is secretly a reward model*." *Advances in Neural Information Processing Systems*, 2023.

Truong, S., Haupt, A., and Koyejo, S. "*Machine Learning from Human Preferences*." *Stanford University*, 2025. [link]

References (2/2)

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., et al. "*Attention Is All You Need*." *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017. [link]

Zhou, C., Liu, P., Xu, P., Iyer, S., Sun, J., et al. "*LIMA: Less Is More for Alignment*." *Advances in Neural Information Processing Systems*, 2023.

Ziegler, D., Stiennon, N., Wu, J., Brown, T., Radford, A., et al. "*Fine-tuning language models from human preferences*." *arXiv preprint arXiv:1909.08593*, 2019.